pravinmishraaws /
Ansible-For-DevOps-Engineers

Code Issues Pull requests Actions Projects Security Insights

Ansible-For-DevOps-Engineers / Section 03: Your First Playbook / Labs
/ Guided Lab 03: Deploy Mini Finance Project using Terraform and Ansible.md

pravinmishraaws Update Guided Lab 03: Deploy Mini Finance Project using Terraform and...

17fb13d · 2 days ago

388 lines (306 loc) · 8.76 KB

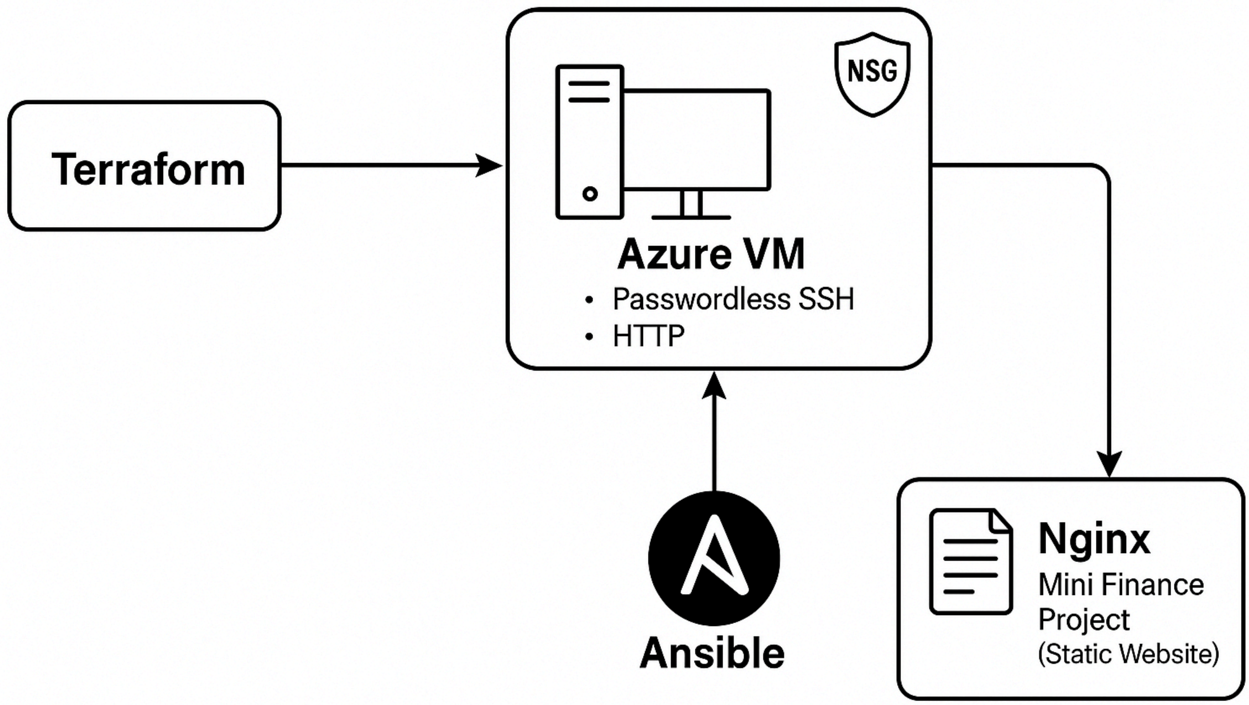
Preview Code Blame Raw

Guided Lab 03: Deploy Mini Finance Project using Terraform and Ansible

Lab Objective

In this hands-on lab, you will:

- Use **Terraform** to provision an Azure VM with passwordless SSH access.
- Allow HTTP (port 80) and SSH (port 22) traffic via a Network Security Group.
- Use **Ansible** (multi-playbook structure) to:
  - Install and configure Nginx web server
  - Clone a static website from GitHub
  - Deploy the site to Nginx web root
  - Restart Nginx gracefully after deployment



Prerequisites

Before starting this lab, ensure:

- **Terraform** is installed (v1.0+)
- **Ansible** is installed
- **Azure CLI** installed and authenticated:

  ```
  az login
  ```

- **SSH keys** exist:
  - Public Key: `~/.ssh/id_rsa.pub`
  - Private Key: `~/.ssh/id_rsa`
- **Git** installed locally.

Lab Environment Setup

Step 1: Create the Project Structure

```
mkdir mini-finance-project
cd mini-finance-project

mkdir terraform ansible
touch terraform/main.tf terraform/variables.tf terraform/terraform.tfva
touch ansible/webserver-installation.yml ansible/inventory.ini
```

✅ You now have a clean structure:

```
mini-finance-project/
├── terraform/
│     ├── main.tf
│     ├── variables.tf
│     ├── terraform.tfvars
│     └── outputs.tf
└── ansible/
      ├── webserver-installation.yml
      └── inventory.ini
```

Terraform Part – Create Infrastructure

Step 2: Write Terraform Files

terraform/variables.tf

```
variable "admin_username" {
  default = "azureuser"
}

variable "location" {
  default = "East US"
}

variable "vm_name" {
  default = "mini-finance-vm"
}
```

```
variable "resource_group_name" {
  default = "mini-finance-rg"
}

variable "ssh_public_key" {
  description = "Path to your SSH Public Key"
  default     = "~/.ssh/id_rsa.pub"
}
```

Explanation:
Define variables to avoid hardcoding names and keep your infrastructure reusable.

terraform/terraform.tfvars

```
admin_username = "azureuser"
```

terraform/main.tf

```
provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name     = var.resource_group_name
  location = var.location
}

resource "azurerm_virtual_network" "vnet" {
  name                = "finance-vnet"
  address_space       = ["10.0.0.0/16"]
  location            = var.location
  resource_group_name = azurerm_resource_group.rg.name
}

resource "azurerm_subnet" "subnet" {
  name                 = "finance-subnet"
  resource_group_name  = azurerm_resource_group.rg.name
  virtual_network_name = azurerm_virtual_network.vnet.name
  address_prefixes     = ["10.0.1.0/24"]
}
```

```
resource "azurerm_network_security_group" "nsg" {
  name                = "finance-nsg"
  location            = var.location
  resource_group_name = azurerm_resource_group.rg.name

  security_rule {
    name                       = "SSH"
    priority                   = 1001
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "22"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }

  security_rule {
    name                       = "HTTP"
    priority                   = 1002
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "80"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }
}

resource "azurerm_public_ip" "pip" {
  name                = "finance-pip"
  location            = var.location
  resource_group_name = azurerm_resource_group.rg.name
  allocation_method   = "Static"
  sku                 = "Standard"
}

resource "azurerm_network_interface" "nic" {
  name                = "finance-nic"
  location            = var.location
  resource_group_name = azurerm_resource_group.rg.name

  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.subnet.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.pip.id
  }
```

```
}

resource "azurerm_network_interface_security_group_association" "nic_nsg
  network_interface_id      = azurerm_network_interface.nic.id
  network_security_group_id = azurerm_network_security_group.nsg.id
}

resource "azurerm_linux_virtual_machine" "vm" {
  name                  = var.vm_name
  resource_group_name   = azurerm_resource_group.rg.name
  location              = var.location
  size                  = "Standard_B1s"
  admin_username        = var.admin_username
  network_interface_ids = [azurerm_network_interface.nic.id]

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
    name                 = "${var.vm_name}-disk"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  admin_ssh_key {
    username   = var.admin_username
    public_key = file(var.ssh_public_key)
  }

  disable_password_authentication = true
}
```

Explanation:
This file provisions an Azure VM with SSH access and allows HTTP + SSH traffic.

terraform/outputs.tf

```
output "public_ip_address" {
  value = azurerm_linux_virtual_machine.vm.public_ip_address
}
```

Explanation:
After applying, it will output the VM's public IP address.

Ansible Part – Configure Web Server

Step 3: Create Inventory

ansible/inventory.ini

```
[web]
<Public_IP_From_Terraform>

[all:vars]
ansible_user=azureuser
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
```

✅ Replace `<Public_IP_From_Terraform>` with the real IP output from Terraform.

Step 4: Write the Multi-Play Ansible Playbook

ansible/webserver-installation.yml

```yaml
- name: Install and Start Nginx Web Server
  hosts: web
  become: yes
  tasks:
    - name: Install Nginx
      apt:
        name: nginx
        state: present
        update_cache: yes

    - name: Start and Enable Nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

```yaml
- name: Deploy Mini Finance Website
  hosts: web
  become: yes
  tasks:
    - name: Clone the Mini Finance project from GitHub
      git:
        repo: "https://github.com/pravinmishraaws/mini_finance.git"
        dest: "/tmp/mini_finance"

    - name: Copy website content to web server root
      copy:
        src: "/tmp/mini_finance/"
        dest: "/var/www/html/"
        owner: www-data
        group: www-data
        mode: "0755"
        remote_src: yes

- name: Configure Nginx to Serve the Site
  hosts: web
  become: yes
  tasks:
    - name: Overwrite default Nginx site configuration
      copy:
        dest: /etc/nginx/sites-available/default
        content: |
          server {
              listen 80;
              server_name _;
              root /var/www/html;
              index index.html;

              location / {
                  try_files $uri /index.html;
              }

              error_page 404 /index.html;
          }
        owner: root
        group: root
        mode: "0644"

- name: Restart Nginx After Configuration
  hosts: web
  become: yes
  tasks:
    - name: Restart Nginx to apply changes
      service:
```

```
        name: nginx
        state: restarted
```

✅ **Explanation**:

- 1st Play: Install and start Nginx
- 2nd Play: Clone and copy application files
- 3rd Play: Restart Nginx after copying files

Execution Steps

Step 5: Deploy Infrastructure

```
cd terraform
terraform init
terraform plan -var-file="terraform.tfvars"
terraform apply -var-file="terraform.tfvars"
```

Confirm when prompted with `yes` .
✅ Note the Public IP output!

Step 6: Configure the Server

Update your inventory file `ansible/inventory.ini` with the VM Public IP.

Run the Ansible playbook:

```
cd ../ansible
ansible-playbook -i inventory.ini webserver-installation.yml
```

✅ Ansible will automatically:

- Install Nginx
- Deploy the website
- Restart Nginx

Verification

- Open your browser and access:

```
http://<Public_IP>
```

✅ You should see the **Mini Finance Project** website!

Conclusion

In this guided lab, you:

- Built cloud infrastructure using **Terraform**.
- Automated web server configuration using **Ansible**.
- Learned industry-standard multi-playbook structure.
- Practiced end-to-end automation, from infrastructure provisioning to application deployment.

🎯 This lab mirrors real-world DevOps and Cloud Engineering workflows.